Quantization in Practice

Apply the quantization algorithms you implemented from scratch to a real language model. Benchmark precision levels, compare GPTQ against naive rounding, study calibration data sensitivity, and produce a deployment recommendation grounded in your own data.

Tooling note. The lab implemented quantization from scratch using only NumPy to build intuition for the underlying mathematics. This assignment uses production quantization libraries (`bitsandbytes`, `AutoGPTQ`, `hqq`) applied to real models. You do **not** use your lab code here - the libraries implement the same algorithms at scale with hardware-optimised kernels. Your lab implementations are the conceptual foundation; the libraries are the engineering tool.

Model Selection

Use the **same model throughout all four tasks**. Choose based on your available hardware:

Model	Params	FP16 size	Hardware
`HuggingFaceTB/SmolLM2-360M`	360M	∼0.7 GB	CPU only
`Qwen/Qwen2.5-0.5B`	500M	∼1.0 GB	CPU only **(recommended)**
`Qwen/Qwen2.5-1.5B`	1.5B	∼3.0 GB	GPU ≥6 GB VRAM
`meta-llama/Llama-3.2-1B`	1B	∼2.0 GB	GPU ≥4 GB VRAM

All experiments in Tasks 1–3 use the same model and the same **evaluation corpus**: the WikiText-2 test split, first 256 tokens of each document (for speed). Perplexity is your primary quality metric throughout.

Download and fix the evaluation samples before running any experiment. Every configuration in Tasks 1–3 must be evaluated on **exactly the same token sequences**. If the tokenisation or sample selection varies between runs, perplexity comparisons are invalid.

Task 1. Precision Benchmarking

Load the chosen model at four precision levels and measure the size–speed–quality trade-off at each level.

Configurations to benchmark

Config	Precision	How to load
FP32	32-bit float	`torch_dtype=torch.float32`
BF16	16-bit bfloat	`torch_dtype=torch.bfloat16`
INT8	8-bit integer	`load_in_8bit=True` (bitsandbytes)
INT4	4-bit integer	`load_in_4bit=True` (bitsandbytes NF4)

Metrics to record for each configuration

1. **Model size on disk** (MB) - saved weights; use `model.get_memory_footprint()` or measure the saved checkpoint.
2. **Peak memory at inference** (MB) - use `torch.cuda.max_memory_allocated()` on GPU, or `tracemalloc` on CPU.
3. **Inference throughput** (tokens/sec) - generate exactly 200 new tokens from a fixed prompt;

average over 3 runs; exclude the first run (warm-up).

4. **Perplexity** on WikiText-2 - compute cross-entropy loss on the fixed evaluation corpus; PPL = exp(mean loss).

Required output

Fill the following table completely:

Precision	Size (MB)	Memory (MB)	Tokens/sec	Perplexity
FP32				
BF16				
INT8				
INT4				

Save: `task1_benchmarks.png` - a 4-panel figure, one subplot per metric, showing value vs precision level.

In your analysis: at which precision does the quality–efficiency trade-off look most favourable? Connect your perplexity results to the quantization errors you observed in the lab's toy-matrix experiments.

Computing perplexity from the model's outputs

```
loss = model(input_ids=tokens, labels=tokens).loss  # mean cross-entropy
perplexity = torch.exp(loss).item()
```

Run this in a `torch.no_grad()` block. Average the loss over multiple samples before calling `exp`.

Task 2. GPTQ vs Naive INT4

In the lab you implemented GPTQ from scratch on an 8×8 matrix. Here you apply it to your full model and quantify how much the Hessian-based error compensation actually helps at real scale.

Configurations to compare

Config	Method	Library
A	BF16 baseline (from Task 1)	-
B	Naive INT4 (from Task 1)	bitsandbytes NF4
C	GPTQ INT4	`AutoGPTQ` or `llm-compressor`

For Config C, calibrate GPTQ on 128 random samples from the WikiText-2 **training split** (not the evaluation split).

Required measurements

- Perplexity on the fixed WikiText-2 evaluation corpus for all three configs.
- Inference throughput (tokens/sec) for B and C.
- GPTQ calibration time (wall-clock minutes).

Weight distribution visualisation

Select **one attention weight matrix** (e.g. the query projection of layer 0). Plot its weight histogram for Config A, B, and C side by side. Your plot should show how GPTQ preserves the distribution shape relative to naive rounding.

Save: `task2_weight_distributions.png` and `task2_gptq_comparison.png` (perplexity and speed bar chart).

In your analysis: express the GPTQ advantage as a percentage perplexity reduction over naive INT4. Explain why GPTQ's column-by-column compensation produces a better weight distribution,

connecting explicitly to the Hessian update rule from the lab.

The 128 calibration samples used to compute the Hessian for GPTQ must come from the WikiText-2 **training split**. Never use evaluation samples for calibration - this is the same data-leakage principle as in Homework 1.

Installing and running AutoGPTQ

```
pip install auto-gptq optimum

from auto_gptq import AutoGPTQForCausalLM, BaseQuantizeConfig

quantize_config = BaseQuantizeConfig(bits=4, group_size=128)
model = AutoGPTQForCausalLM.from_pretrained(model_name, quantize_config)
model.quantize(calibration_data)   # list of tokenised samples
model.save_quantized("gptq_model")
```

Task 3. Calibration Data Sensitivity

GPTQ's quality depends on the calibration data used to estimate the Hessian. This task investigates whether it matters what domain the calibration data comes from.

Calibration corpora

Run GPTQ three times - identical hyperparameters, identical model, different calibration sets only:

Run	Calibration corpus	Source
C1	General Wikipedia text	`wikimedia/wikipedia` (`20231101.en`)
C2	Python code	`codeparrot/github-code` filtered for Python
C3	Formal / academic text	`scientific_papers` (arXiv split) or `pg19`

Use 128 samples for each calibration run. Save each quantized model to disk so you do not need to re-run calibration.

Evaluation corpora

Evaluate all three GPTQ variants on **two** test sets:

1. **WikiText-2 test** - general English prose (same as Tasks 1–2).
2. **Python code perplexity** - 50 short Python functions from the `codeparrot/github-code` test split.

Required output

Fill the following table:

Config	Calibration domain	WikiText-2 PPL	Python code PPL
BF16 baseline	-		
GPTQ C1	Wikipedia		
GPTQ C2	Python		
GPTQ C3	Academic		

Save: `task3_calibration_study.png` - a grouped bar chart showing both evaluation domains for all four configurations.

In your analysis (≥300 words): does calibration domain match matter? Which configuration

performs best on WikiText-2, and which on Python code? Under what real-world deployment conditions would you invest in domain-matched calibration data?

Why calibration data matters

GPTQ estimates the Hessian $H = 2XX^T$ from the activations produced by the calibration inputs. If the calibration inputs activate a different set of attention patterns and outlier channels than the deployment data, the error-compensation step corrects for the wrong signal. This is the real-model analogue of the ordering study in the lab's Extension 3.

Task 4. Deployment Recommendation Report

Write a structured analysis that uses your experimental results from Tasks 1–3 to make concrete, justified recommendations for three deployment scenarios.

Scenarios

1. **Edge / CPU deployment.** A lightweight local assistant running on a laptop with no dedicated GPU. Constraints: $\leq$4 GB total RAM for the model, latency $<$2 seconds per response.
2. **Server inference, throughput-optimised.** A single GPU server (e.g. A100 40 GB) serving hundreds of requests per minute. Constraint: maximise tokens/sec; quality degradation $<$5% relative perplexity increase.
3. **Domain-specific deployment.** A code assistant fine-tuned on proprietary Python codebases, deployed on a single consumer GPU (RTX 3080, 10 GB VRAM). Constraint: must maintain good code perplexity after quantization.

Requirements

For each scenario, your recommendation must:

- State the recommended precision and quantization method.
- Cite at least two specific numbers from your Tasks 1–3 results to justify the choice.
- Acknowledge the trade-off that was made (what you gave up to gain what).
- State whether domain-matched calibration data is worth the extra effort for this scenario, and why.

Minimum length: 500 words total across the three scenarios. Write this in a dedicated `report.md` file committed to the repository.

Bonus

Bonus A - 2-bit quantization.

Extend your Task 2 comparison by adding a 2-bit configuration using `HQQ` (Half-Quadratic Quantization). HQQ requires no calibration data, supports 2-bit natively, and runs on CPU and consumer GPUs.

1. Install HQQ: `pip install hqq`.
2. Quantize the same model from Tasks 1–3 to 2-bit using HQQ.
3. Add a `INT2 (HQQ)` row to your Task 1 benchmark table (size, memory, throughput, perplexity).
4. Plot the full precision curve: FP32 $\rightarrow$ BF16 $\rightarrow$ INT8 $\rightarrow$ INT4 $\rightarrow$ INT2, showing perplexity vs model size. Identify the quality floor - the point where perplexity degrades sharply.

In your analysis: connect HQQ's 2-bit results to the Hadamard rotation you implemented in the lab. The core idea behind QuIP# (the leading research method for 2-bit quantization) is exactly the incoherence transformation you studied - Hadamard rotation makes weights more uniform so

that 2-bit rounding loses less information. Discuss whether the perplexity you observe at 2-bit is consistent with that theoretical expectation.

Save: `bonus_2bit_precision_curve.png`

Bonus B - Cross-assignment collaboration.

Collaborate with a classmate who completed a different Homework 3 topic. Take a model or system from their assignment, apply your quantization pipeline to it, and jointly report the results.

Compatible assignments - the tools used in this assignment (`bitsandbytes`, `AutoGPTQ`, `hqq`) work with **HuggingFace-compatible transformer language models**. Only the following topics produce a quantizable model in this sense:

Topic	Model to quantize	Condition
LLM from Scratch	The DPO fine-tuned model	Always compatible. Best pairing.
Generative AI	The generator LLM inside the RAG pipeline	Compatible if a local open-weight model was used.
Agentic AI	The LLM backbone of the agent	Compatible only if a local open-weight model was used - not an API model.
RAG	The generator LLM	Compatible if a local open-weight model was used.

Reinforcement learning policy networks (DQN, A2C) are tiny PyTorch MLPs - quantizing them with LLM tools is meaningless. Computer Vision CNNs and CLIP models use a completely different quantization toolchain (`torch.quantization`) that is outside the scope of this assignment. Do not attempt to use these as the provider model.

What each student does:

- **Model provider** (your classmate): shares a saved model checkpoint, the inference code needed to run it, and a small evaluation set (at minimum: 10 example inputs with expected outputs).
- **Quantization student** (you): applies at least two precision configurations from your pipeline (e.g. BF16 and GPTQ INT4), runs the evaluation, and reports quality before and after quantization.

Joint deliverable: a `collaboration_report.md` (committed to both repositories) containing: the topic and model provided, the quantization method applied, the evaluation results, and a brief discussion of whether the model's task performance survived quantization.

Both students receive the bonus points independently - one for providing, one for quantizing. There is no penalty if the collaboration does not work out; this is a bonus task.

Deliverables

Submit a GitHub repository. All items below must be present and committed.

File / Folder	Content
`task1_benchmark.py` (or `.ipynb`)	Precision sweep, metrics collection, Task 1 table and plot
`task2_gptq.py` (or `.ipynb`)	GPTQ vs naive INT4 comparison, weight distribution plots
`task3_calibration.py` (or `.ipynb`)	Three GPTQ runs with different calibration corpora, cross-evaluation
`report.md`	Task 4 deployment recommendation ($\geq$500 words)
`results/task1_benchmarks.png`	4-panel benchmark figure
`results/task2_weight_distributions.png`	Weight histograms for BF16 vs INT4 vs GPTQ
`results/task2_gptq_comparison.png`	Perplexity and throughput comparison
`results/task3_calibration_study.png`	Grouped bar chart for calibration study
`requirements.txt`	All dependencies with versions
`README.md`	See requirements below
`results/bonus_2bit_precision_curve.png`	Precision curve FP32 $\rightarrow$ INT2 (Bonus A)
`collaboration_report.md`	Joint report with collaborating classmate (Bonus B)

Do not commit quantized model weights. They are large and reproducible from your scripts. Commit only the code and results.

GPTQ calibration takes several minutes per run even on small models. Save every quantized model to disk after the first calibration and load it for subsequent evaluations. Never re-run calibration just to re-measure perplexity.

README requirements

1. **Model and hardware** - which model you chose and what hardware you ran on (CPU/GPU, RAM/VRAM).
2. **Task 1 results** - the completed benchmark table.
3. **Task 2 results** - GPTQ perplexity advantage over naive INT4 as an absolute and percentage reduction.
4. **Task 3 results** - the completed calibration study table and your conclusion on calibration domain sensitivity.
5. **Execution instructions** - one command per task to reproduce all results from a clean environment.